CS509 - Assignment 1

Single Task: GEMM | Buddy Tasks: BFS, DFS and SSSP

Purpose. This document defines the algorithms, text-file input formats, expected outputs, and timing requirements for Assignment 1. The driver program must read the input file, prepare the required data structure, call the algorithm, and print both the final result and the algorithm execution time.

Timing rule. Measure only the algorithm execution time. File reading, input parsing, adjacency-list-to-CSR conversion, output printing, and other setup work must not be included.

1. Assignment Structure

Task Type	Algorithms	Work Mode
Single task	GEMM - simple implementation and blocking implementation, CSR graph implementation	Individual
Buddy tasks	BFS, DFS and SSSP with positive edge weights only	Pair of two students

2. Brief Explanation of the Algorithms

2.1 GEMM - General Matrix Multiplication

GEMM computes the product of two matrices. For matrices A of size M x K and B of size K x N, the output matrix C has size M x N. Each element C[i][j] is obtained by multiplying corresponding values from row i of A and column j of B and adding them.

- Simple GEMM: uses the direct nested-loop implementation.
- Blocking GEMM: divides matrices into smaller blocks or tiles so that data is reused more efficiently in cache.
- Both implementations must produce the same result matrix for the same input.

2.2 CSR Graph

CSR (Compressed Sparse Row) is a compact format used to store sparse graphs or matrices efficiently.

For a graph, the adjacency list is converted into three arrays:

- **row_ptr**: tells where each vertex's neighbour list starts and ends.
- **col_idx**: stores the neighbouring vertex numbers.
- **values**: stores edge weights, when the graph is weighted.

2.3 BFS - Breadth-First Search

BFS visits graph vertices level by level from a given source vertex. It uses a queue. For an unweighted graph, BFS can also report the minimum number of edges from the source to every reachable vertex.

2.4 DFS - Depth-First Search

DFS explores one path as deeply as possible before returning to explore another path. It may be implemented recursively or with an explicit stack. The output should contain a valid DFS traversal beginning from the given source vertex.

2.5 SSSP - Single-Source Shortest Path

SSSP computes the shortest distance from one source vertex to all other vertices. For this assignment, all edge weights must be positive. A suitable positive-weight shortest-path algorithm, such as Dijkstra's algorithm, should be used.

3. General Input-File Rules

- Every test case must be stored in a separate .txt file.
- Use space-separated integer values unless otherwise stated.
- Vertex numbering must be consistent throughout the semester. The recommended convention is 0 to V-1.
- For graph inputs, the file must store the graph as an adjacency list.
- The driver reads the adjacency-list file and then calls the provided/helper conversion function to create CSR input before calling the graph algorithm.
- The CSR conversion time must not be included in the reported algorithm execution time.
- The required graph sizes are 10, 100, 10,000, 50,000 and 100,000 vertices.

4. CSR Graph

4.1 Graph Input Format and CSR Conversion

- The input format for graph problems will be an adjacency list.
- A helper function must be provided to convert the adjacency-list representation into Compressed Sparse Row (CSR) format whenever CSR-based input is required by the implementation.
- CSR will be explained and the required conversion approach before students are expected to use it.
- The adjacency-list-to-CSR conversion is preprocessing. Its execution time must not be included in the reported algorithm runtime.
- For an algorithm that uses CSR, timing must begin only after the CSR representation has been prepared.

4.2 Required Graph Input Sizes

Graph assignments must include test inputs having the following numbers of vertices:

Test Scale	**Number of Vertices (V)**
Very small	10
Small	100
Medium	10,000
Large	50,000
Very large	100,000

The number of edges (E), graph type, and other relevant properties must also be recorded for each graph test case in the README.

5. GEMM Input and Output Format

5.1 GEMM Text-File Input Format

Use the same input file for both the simple and blocking implementations so that their results and execution times can be compared fairly.

```
M K N
A row 0 values
A row 1 values
...
A row M-1 values
B row 0 values
B row 1 values
...
B row K-1 values
```

Meaning: A is M x K, B is K x N, and the result C is M x N. Each matrix row is written on a separate line.

5.2 GEMM Example Input

```
2 3 2
1 2 3
4 5 6
7 8
9 10
11 12
```

This represents A = [[1,2,3],[4,5,6]] and B = [[7,8],[9,10],[11,12]].

5.3 Expected GEMM Output

The program must print the result matrix and the algorithm execution time. The simple and blocking versions must be reported separately.

```
Algorithm: GEMM Simple
Result matrix:
58 64
139 154
Execution time: <value> ms

Algorithm: GEMM Blocking
Result matrix:
58 64
139 154
Execution time: <value> ms
```

5.4 Matrix Problem Inputs

- Matrix problems must also use separate test files, with one test case per file.

- For each matrix test, the README must record the input type and the applicable dimensions, such as rows x columns or M x K and K x N.
- Any assignment-specific matrix sizes or formats will be provided with the corresponding assignment instructions.

6. BFS and DFS Input and Output Format

6.1 Unweighted Adjacency-List File Format

Use the following format for BFS and DFS:

```
V E
u0 degree neighbor1 neighbor2 ...
u1 degree neighbor1 neighbor2 ...
...
u(V-1) degree neighbor1 neighbor2 ...
SOURCE s
```

- V: number of vertices.
- E: number of edges. For an undirected graph, count each graph edge once in E even though it appears in both adjacency lists.
- u: vertex whose adjacency list is being written.
- degree: number of neighbours listed after u.
- s: source vertex for BFS or DFS.
- For a vertex with no neighbours, write: u 0.

6.2 BFS/DFS Example Input

```
5 5
0 2 1 2
1 2 0 3
2 3 0 3 4
3 2 1 2
4 1 2
SOURCE 0
```

6.3 Expected BFS Output

Print the traversal from the source. If distances are also implemented, print the minimum edge-count distance for every vertex. Unreachable vertices should be shown clearly, for example as INF or -1.

```
Algorithm: BFS
Source: 0
Traversal: 0 1 2 3 4
Distances:
0 0
1 1
2 1
3 2
4 2
Execution time: <value> ms
```

6.4 Expected DFS Output

Print a valid DFS traversal beginning from the source. The exact order may depend on the order of neighbours in the input file, but it must be valid and reproducible for the same file.

```
Algorithm: DFS
Source: 0
Traversal: 0 1 3 2 4
Execution time: <value> ms
```

7. SSSP Input and Output Format

7.1 Positive-Weighted Adjacency-List File Format

For SSSP, each neighbour must be followed by its positive edge weight.

```
V E
u0 degree neighbor1 weight1 neighbor2 weight2 ...
u1 degree neighbor1 weight1 neighbor2 weight2 ...
...
u(V-1) degree neighbor1 weight1 neighbor2 weight2 ...
SOURCE s
```

- Every weight must be greater than 0.
- For a directed graph, list only outgoing edges.
- For an undirected graph, include each edge in the adjacency list of both endpoint vertices.
- For a vertex with no outgoing neighbours, write: u 0.

7.2 SSSP Example Input

```
5 6
0 2 1 4 2 1
1 1 3 1
2 2 1 2 3 5
3 1 4 3
4 0
SOURCE 0
```

7.3 Expected SSSP Output

Print the shortest distance from the source to every vertex. Unreachable vertices should be displayed as INF or another clearly documented value. Predecessor values or reconstructed paths may also be printed, but distances are mandatory.

```
Algorithm: SSSP
Source: 0
Vertex  Distance
0       0
1       3
2       1
3       4
4       7
Execution time: <value> ms
```

8. Timing and Measurement Requirements

- Start the timer immediately before calling the algorithm.
- Stop the timer immediately after the algorithm finishes.
- Do not include file reading, input parsing, memory allocation done as setup, adjacency-list-to-CSR conversion, result printing, or file writing.
- Report time in milliseconds or seconds. Use the same unit consistently within a comparison table.
- For very fast inputs, repeated runs may be used and the average may be reported, provided the number of runs is documented.
- Run both GEMM implementations on the same matrix inputs and report their times separately.
- For every BFS, DFS and SSSP input file, report the algorithm time for that particular graph.

9. Required README Result Tables

The repository README must contain a completed table for every assignment test case. Add or remove rows as required.

9.1 GEMM Results Table

Test File	Input Type / Size	Expected Output	Actual Output	Simple Time	Blocking Time	Block Size	Status
gemm_test_01.txt	M x K and K x N	Result matrix	Result matrix	... ms	... ms	...	Pass/Fail
...	...	...	...	...	...	...	...

9.2 Graph Results Table

Algorithm	Test File	Vertices	Edges	Input Type	Source	Expected Output	Actual Output	Time	Status
BFS	bfs_10.txt	10	...	Unweighted adjacency list	...	Traversal / distances	...	... ms	Pass/Fail
DFS	dfs_10.txt	10	...	Unweighted adjacency list	...	Traversal	...	... ms	Pass/Fail
SSSP	sssp_10.txt	10	...	Positive weighted adjacency list	...	Shortest distances	...	... ms	Pass/Fail
...	...	100 / 10000 / 50000 / 100000	...	...	...	...	...	...	...

10. Suggested Test-File Naming

Algorithm	Suggested Names
GEMM	gemm_test_01.txt, gemm_test_02.txt, ...
BFS	bfs_10.txt, bfs_100.txt, bfs_10000.txt, bfs_50000.txt, bfs_100000.txt

Algorithm	Suggested Names
DFS	dfs_10.txt, dfs_100.txt, dfs_10000.txt, dfs_50000.txt, dfs_100000.txt
SSSP	sssp_10.txt, sssp_100.txt, sssp_10000.txt, sssp_50000.txt, sssp_100000.txt

11. Minimum Expected Driver Behaviour

- Accept the selected algorithm and input-file path through the common wrapper/menu or terminal arguments.
- Read and validate the input file.
- For graph tasks, construct the adjacency list and call the helper function to convert it to CSR.
- Call the selected algorithm.
- Print the final answer in a readable form.
- Print only the measured algorithm execution time as the reported timing.
- Return a clear error message for an invalid or missing input file.